36.3 Android 节点应用

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~185k tokens，输出 ~5k tokens（本节）

Android 应用与 iOS 应用一样扮演"节点"角色——连接到 Gateway 并暴露设备的硬件能力。它使用 **Kotlin + Jetpack Compose** 构建，是三个原生客户端中唯一运行在非 Apple 平台上的。

36.3.1 Kotlin 架构

项目配置

Android 应用位于 `apps/android/`，使用 Gradle 构建：

```
// apps/android/app/build.gradle.kts
android {
    namespace = "ai.openclaw.android"
    compileSdk = 36
    defaultConfig {
        applicationId = "ai.openclaw.android"
        minSdk = 31        // Android 12 (S) 最低
        targetSdk = 36
    }
    buildFeatures {
        compose = true      // Jetpack Compose UI
        buildConfig = true
    }
}
```

`assets.srcDir` 指向了共享库的资源目录——Android 应用直接复用 `OpenClawKit` 的资源文件（如音效、图标），但由于 Kotlin 无法直接使用 Swift 包，协议和模型需要在 Kotlin 中重新实现。

源码结构

```
apps/android/app/src/main/java/ai/openclaw/android/
├── MainActivity.kt              # Activity 入口
├── MainViewModel.kt             # ViewModel 状态管理
├── NodeApp.kt                   # Application 子类
├── NodeRuntime.kt               # 核心运行时（类似 iOS 的 NodeAppModel）
├── NodeForegroundService.kt     # 前台服务（保持连接）
├── SecurePrefs.kt               # 加密偏好存储
├── SessionKey.kt                # 会话键
├── gateway/                     # WebSocket 连接 + 服务发现
│   ├── GatewaySession.kt
│   ├── GatewayDiscovery.kt
│   ├── DeviceAuthStore.kt
│   └── DeviceIdentityStore.kt
├── node/                        # 设备能力实现
│   ├── CanvasController.kt
│   ├── CameraCaptureManager.kt
│   ├── ScreenRecordManager.kt
│   ├── LocationCaptureManager.kt
│   └── SmsManager.kt
├── protocol/                    # Gateway 协议类型
│   ├── OpenClawCapability.kt
│   ├── OpenClawCameraCommand.kt
│   ├── OpenClawCanvasCommand.kt
│   └── OpenClawScreenCommand.kt
├── chat/                        # 聊天功能
├── voice/                       # 语音唤醒 + Talk Mode
│   ├── VoiceWakeManager.kt
│   └── TalkModeManager.kt
└── ui/                          # Compose UI
```

NodeRuntime 核心

`NodeRuntime` 是 Android 应用的核心运行时，聚合了所有设备能力：

```kotlin
// apps/android/app/src/main/java/ai/openclaw/android/NodeRuntime.kt
class NodeRuntime(context: Context) {
    private val scope = CoroutineScope(SupervisorJob() + Dispatchers.IO)

    val prefs = SecurePrefs(appContext)
    val canvas = CanvasController()
    val camera = CameraCaptureManager(appContext)
    val location = LocationCaptureManager(appContext)
    val screenRecorder = ScreenRecordManager(appContext)
    val sms = SmsManager(appContext)

    private val voiceWake: VoiceWakeManager by lazy {
        VoiceWakeManager(context = appContext, scope = scope,
            onCommand = { command -> ... })
    }
}
```

与 iOS 的 `NodeAppModel` 对应， `NodeRuntime` 创建并持有所有能力管理器的实例。它使用 Kotlin 协程（ `CoroutineScope + SupervisorJob` ）进行异步操作。

> **衍生解释 — Kotlin 协程与 SupervisorJob**
>
> Kotlin 协程是 Kotlin 的异步编程方案，类似于 Swift 的 async/await。 `CoroutineScope` 定义了协程的生命周期范围， `SupervisorJob` 确保一个子协程的失败不会取消其他兄弟协程——这对于节点运行时很重要，因为摄像头失败不应该影响位置服务。

前台服务

Android 要求长时间运行的后台任务必须通过 **前台服务**（Foreground Service）实现。 `NodeForegroundService` 在 `MainActivity.onCreate` 时启动：

```kotlin
// MainActivity.kt
override fun onCreate(savedInstanceState: Bundle?) {
    NodeForegroundService.start(this)  // 启动前台服务
    // ...
}
```

前台服务会在通知栏显示一个持久通知，告知用户 OpenClaw 正在运行。这确保了即使用户切换到其他应用，WebSocket 连接也不会被系统杀死。

36.3.2 Canvas / 摄像头 / 屏幕录制

Android 节点暴露的设备能力与 iOS 对齐：

能力	管理器	Android API
Canvas	`CanvasController`	WebView（可选）
摄像头	`CameraCaptureManager`	Camera2 API
屏幕录制	`ScreenRecordManager`	MediaProjection API
位置	`LocationCaptureManager`	LocationManager
SMS	`SmsManager`	SmsManager API

每个能力管理器都需要与 Android 的权限系统集成。 `PermissionRequester` 统一处理运行时权限请求：

```
// MainActivity.kt
permissionRequester = PermissionRequester(this)
viewModel.camera.attachPermissionRequester(permissionRequester)
viewModel.sms.attachPermissionRequester(permissionRequester)
viewModel.screenRecorder.attachScreenCaptureRequester(screenCaptureRequester)
```

屏幕录制比较特殊：Android 的 `MediaProjection` API 不仅需要运行时权限，还需要用户在系统弹窗中明确同意，因此有独立的 `ScreenCaptureRequester` 。

`MainActivity` 还监听 `preventSleep` 状态流，在需要时设置 `FLAG_KEEP_SCREEN_ON` 防止屏幕自动关闭——这在 Canvas 展示或对话模式下很重要。

36.3.3 可选 SMS 支持

Android 独有的能力是 **SMS 短信**支持。当用户授予 SMS 权限后，AI Agent 可以通过 `SmsManager` 读取和发送短信。这在 iOS 上由于系统限制不可能实现。

SMS 能力的声明和使用遵循与其他能力相同的模式：在连接 Gateway 时通过 capabilities 声明，Gateway 据此决定是否暴露 SMS 相关工具。

```
OpenClawCapability:
  - canvas
  - camera
  - screen
  - voiceWake
  - location
  - sms          ← Android 独有
```

本节小结

1. **Android 应用使用 Kotlin + Jetpack Compose + Gradle** 构建，最低支持 Android 12（API 31）。
2. **NodeRuntime 是核心运行时**，使用 Kotlin 协程和 SupervisorJob 管理各设备能力管理器。
3. **前台服务**确保 WebSocket 连接在后台不被系统终止，通过持久通知告知用户。
4. **设备能力**（Canvas、摄像头、屏幕录制、位置、SMS）各自有独立的管理器，统一通过 PermissionRequester 处理权限。
5. **SMS 是 Android 独有能力**，使得 AI Agent 可以读写短信。